

FIR-BASED DECIMATION AND INTERPOLATION USING XILINX FIR COMPILER IP

Project Submission Report

Submitted By

KARANAM UDAYKUMAR

Submission For

Apex Plus – FPGA Design Assessment (Round 1)

Project Overview

This project implements a multirate digital signal processing system that performs decimation by 3 followed by interpolation by 12 on a 16-bit Q1.15 fixed-point input signal. The design utilizes Xilinx FIR Compiler IP cores to realize both the decimator and interpolator using 51-tap low-pass FIR filters with a cutoff frequency of 15 MHz.

The implementation was developed and verified using Verilog HDL and Xilinx Vivado. Python was used for generating the input sine-wave stimulus, fixed-point conversion, FFT analysis, and visualization of the processed signals.

Tools Used

- Xilinx Vivado Design Suite
- Verilog HDL
- Xilinx FIR Compiler IP
- Python 3.x
- NumPy
- Matplotlib

FIR-BASED DECIMATION AND INTERPOLATION USING XILINX FIR COMPILER IP

1. Introduction

A **Finite Impulse Response (FIR) filter** is a digital filter widely used in signal processing applications because of its stability, linear phase response, and ease of implementation. FIR filters are commonly employed in noise reduction, audio and speech processing, image enhancement, and communication systems. Unlike recursive filters, the output of an FIR filter depends only on the present and past input samples, ensuring predictable and reliable performance. These characteristics make FIR filters an essential component in modern digital signal processing (DSP) systems.

This project focuses on the design and implementation of an FIR filter using **Verilog Hardware Description Language (HDL)** for FPGA-based hardware realization. FPGAs provide high-speed processing, parallel computation, and flexibility, making them suitable for real-time DSP applications. The FIR filter is implemented using a series of multipliers, adders, and delay elements to perform the required filtering operation. Through simulation and hardware implementation, the project demonstrates how Verilog HDL can be used to develop an efficient and reliable FIR filter architecture for practical signal processing applications.

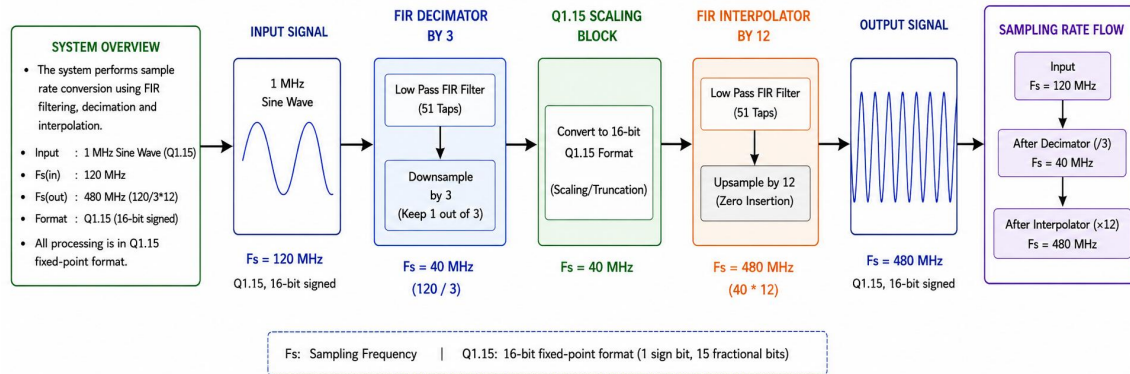
2. Objective

The primary objective of this project is to design and implement a **Finite Impulse Response (FIR) filter** using **Verilog HDL** and verify its functionality through simulation in **Xilinx Vivado**. The project aims to develop a hardware-efficient FIR filter architecture using delay elements, multipliers, and adders to perform digital signal filtering. By implementing the filter on an FPGA design platform, the project demonstrates the practical application of digital signal processing concepts in hardware.

Another objective is to utilize **Python** for generating input test signals, converting data into the required format, and analyzing the filter performance using **Fast Fourier Transform (FFT)**. Python is used to create sample input data and process the simulation results obtained from Vivado. The project also aims to validate the filter operation by comparing the input and output signals in both time and frequency domains, thereby providing a comprehensive understanding of FIR filter design, simulation, and analysis.

3. Block Diagram

BLOCK DIAGRAM – DECIMATION BY 3 AND INTERPOLATION BY 12



The block diagram represents an FPGA-based multirate digital signal processing system that performs **decimation by 3** followed by **interpolation by 12** on a 1 MHz sine wave signal. The entire system operates using **Q1.15 fixed-point format** and utilizes FIR filters to prevent aliasing and imaging effects during sample-rate conversion.

1. Input Signal

The input to the system is a **1 MHz sine wave** represented in **16-bit Q1.15 fixed-point format**. The signal is sampled at a frequency of **120 MHz** and serves as the test input for the DSP processing chain.

2. FIR Decimator by 3

The first processing block is a **51-tap FIR low-pass filter** followed by a downsampler. The FIR filter removes high-frequency components that could cause aliasing during decimation. After filtering, the downsampler keeps one sample out of every three samples, reducing the sampling frequency from **120 MHz to 40 MHz**.

3. Q1.15 Scaling Block

The output of the FIR decimator typically has a larger bit width due to multiplication and accumulation operations inside the filter. The scaling block converts this wider output back to **16-bit Q1.15 format** through scaling or truncation. This ensures consistent fixed-point representation throughout the system and prevents overflow.

4. FIR Interpolator by 12

The scaled signal is then passed to a **51-tap FIR interpolator**. The interpolator first inserts eleven zeros between consecutive samples, effectively increasing the sampling rate by a factor of twelve. A low-pass FIR filter then removes the unwanted spectral

images introduced by zero insertion and reconstructs the signal. As a result, the sampling frequency increases from **40 MHz to 480 MHz**.

5. Output Signal

The final output is a filtered and reconstructed **1 MHz sine wave** represented in **Q1.15 format** with a sampling frequency of **480 MHz**. The signal maintains its original frequency content while operating at a much higher sampling rate.

6. Sampling Rate Conversion Summary

Stage	Operation	Sampling Frequency
Input Signal	Original Signal	120 MHz
After Decimator	Downsample by 3	40 MHz
After Interpolator	Upsample by 12	480 MHz

4. FIR Filter Theory

A **Finite Impulse Response (FIR) filter** is a type of digital filter whose impulse response settles to zero in a finite number of sampling periods. The output of an FIR filter is obtained by multiplying the current and previous input samples with a set of predefined coefficients and then summing the results. Since FIR filters do not use feedback elements, they are inherently stable and provide a linear phase response, making them suitable for applications that require minimal signal distortion.

Mathematically, the output of an FIR filter is expressed as:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k] \quad y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where $y[n]$ is the output signal, $x[n-k]$ represents the current and delayed input samples, $h[k]$ denotes the filter coefficients, and N is the number of taps in the filter. Each tap consists of a delay element, a multiplier, and an adder. By appropriately selecting the filter coefficients, FIR filters can be designed as low-pass, high-pass, band-pass, or band-stop filters. Due to their simple structure, stability, and ease of implementation in hardware, FIR filters are widely used in FPGA- and ASIC-based digital signal processing systems.

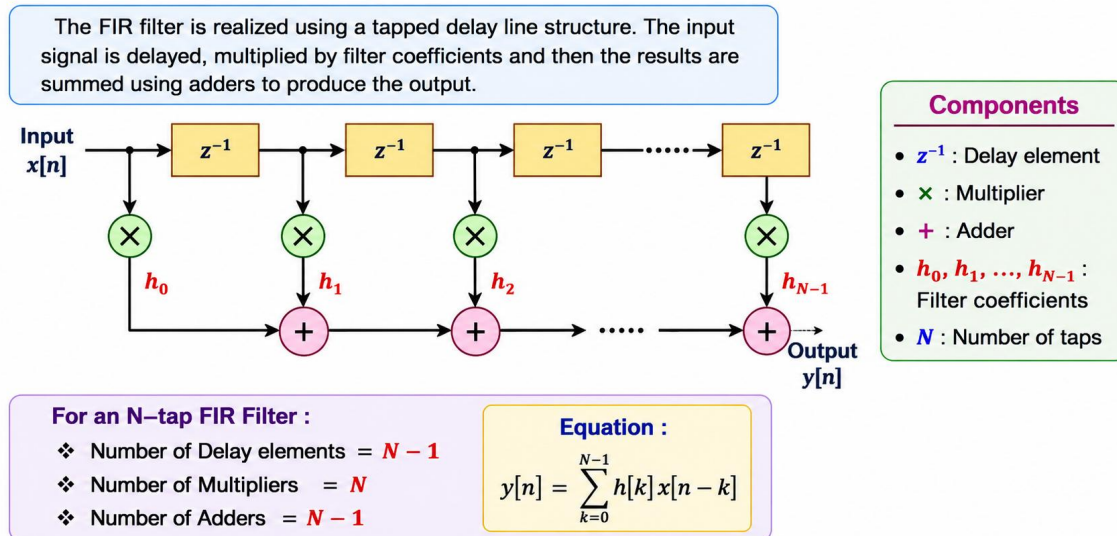
5. FIR Architecture

The architecture of a **Finite Impulse Response (FIR) filter** consists of three main components: **delay elements, multipliers, and adders**. The input signal passes through a series of delay registers that store previous input samples. Each delayed sample is multiplied by a corresponding filter coefficient, and the resulting products are summed together to generate the filter output. This structure is commonly known as the **tapped delay line architecture**.

For an N -tap FIR filter, there are $N-1$ delay elements, N multipliers, and N adders. The filter coefficients determine the frequency response and filtering characteristics of the system. In FPGA implementations, the architecture can be efficiently realized using registers for delays, dedicated multiplier blocks, and adder trees for accumulation. The parallel nature of FPGA hardware allows multiple multiplication and addition operations to be performed simultaneously, resulting in high-speed processing suitable for real-time digital signal processing applications.

Basic FIR Filter Architecture:

4. FIR ARCHITECTURE



6. Verilog Implementation

The FIR filter is implemented in **Verilog HDL** using a combination of delay registers, multipliers, and adders. The input samples are stored in a shift register structure to create the required delays. Each delayed sample is multiplied by a predefined filter coefficient, and the products are summed to generate the filtered output. The design is synchronized using a clock signal, allowing the filter to process one sample per clock cycle.

In this project, the FIR filter coefficients are defined as constant values within the Verilog code. On every rising edge of the clock, the input data is shifted through the delay registers, and the multiply-and-accumulate (MAC) operation is performed to calculate the output. The design is then verified using a testbench that applies different input values and observes the resulting output waveform. Simulation in Vivado confirms the correct operation of the FIR filter and demonstrates its effectiveness in digital signal processing applications.

VERILOG CODE (TOP MODULE)

```
module top_module(  
  
    input clk,  
  
    input rst,  
  
    input signed [15:0] data_in,  
  
    input data_valid,  
  
  
    output signed [15:0] data_out,  
  
    output data_out_valid,  
  
    output signed [15:0] int_out,  
  
    output signed [23:0] dec_out  
  
);  
  
// wires for decimator output  
  
wire signed [23:0] dec_out;  
  
wire dec_valid;  
  
// scaled signal  
  
wire signed [15:0] scaled_data;  
  
// wires for interpolator output  
  
wire signed [23:0] int_out;  
  
wire int_valid;  
  
wire ready1;
```

```

wire ready2;

wire dec_ready;

assign dec_ready= 1'b1;

assign int_ready=1'b1;

// FIR DECIMATOR

fir_decimator my_decimator (

    .aclk(clk),

    .aclken(1'b1),

    .aresetn(~rst),

    .s_axis_data_tvalid(data_valid),

    .s_axis_data_tready(ready1),

    .s_axis_data_tdata(data_in),

    .m_axis_data_tvalid(dec_valid),

    .m_axis_data_tready(dec_ready),

    .m_axis_data_tdata(dec_out)

);

// Scaling to Q1.15

assign scaled_data = dec_out[23:8];

// FIR INTERPOLATOR

fir_interpolator my_interpolator (

    .aclk(clk),

    .s_axis_data_tvalid(dec_valid),

    .s_axis_data_tready(ready2),

    .s_axis_data_tdata(scaled_data),

    .m_axis_data_tvalid(int_valid),

```

```

        .m_axis_data_tdata(int_out)

    );

    // Final Output

    assign data_out = dec_out[15:0];

    assign data_out_valid = dec_valid;

    // Debug print

    always@ ( posedge clk) begin

        if(dec_valid)

            $display("dec_out=%d",dec_out);

    end

endmodule

```

7. Testbench

A **testbench** is used to verify the functionality of the FIR filter design before hardware implementation. It provides the necessary input signals, clock generation, and reset conditions to simulate the behavior of the FIR filter. By applying different input values and observing the corresponding outputs, the correctness of the filter operation can be validated. The testbench does not contain any hardware-specific logic and is used solely for simulation purposes.

In this project, the testbench generates a periodic clock signal and applies a sequence of input samples to the FIR filter. The simulation is performed using **Vivado Simulator**, where the output waveform is analyzed to ensure that the filter produces the expected results. The testbench helps identify design errors, verify timing behavior, and confirm that the FIR filter operates correctly under different input conditions.

TECHBENCH CODE

```

timescale 1ns/1ps

module tb_top;

    reg clk;

    reg rst;

    reg signed [15:0] data_in;

```



```

reg data_valid;

wire signed [15:0] data_out;

wire data_out_valid;

wire signed [15:0] int_out;

wire signed [23:0] dec_out;

integer infile;

integer outfile;

integer r;

top_module uut (

    .clk(clk),

    .rst(rst),

    .data_in(data_in),

    .data_valid(data_valid),

    .data_out(data_out),

    .data_out_valid(data_out_valid),

    .int_out(int_out),

    .dec_out(dec_out)

);

// Clock Generation

initial begin

    clk = 0;

    forever #5 clk = ~clk;

end

// Main Test

initial begin

```

```

    clk=0;

    rst = 1;

    data_in=0;

    data_valid=0;

    #100;

    rst = 0;

    data_valid=1;

    infile = $fopen("input_signal.txt","r");

    if(infile == 0)

        $display("INPUT FILE NOT OPENED");

    else

        $display("INPUT FILE OPENED SUCCESSFULLY");

        outfile = $fopen("output_signal.txt","w");

    if(infile == 0) begin

        $display("ERROR: Input file not found");

        $finish;

    end

    if(outfile == 0) begin

        $display("ERROR: Output file not created");

        $finish;

    end

    while(!$feof(infile)) begin

        @(posedge clk);

        r = $fscanf(infile,"%d\n",data_in);

        data_valid = 1;

```

```

end

@(posedge clk)

data_valid = 0;

repeat(200)@(posedge clk);

$fclose(infile);

$fclose(outfile);

$stop;

end

// Store Output

always @(posedge clk) begin

    if(data_out_valid) begin

        // console print

        $display("dec_out=%0d int_out=%0d data_out=%0d",

            dec_out, int_out, data_out);

        // FILE write (THIS LINE GOES HERE)

        $fwrite(outfile, "%0d %0d %0d\n",

            dec_out, int_out, data_out);

    end

end

endmodule

```

8. Simulation using Vivado

The designed FIR decimation and interpolation system was simulated using **Xilinx Vivado Design Suite** to verify its functionality and performance. The Verilog modules for the FIR decimator, Q1.15 scaling block, and FIR interpolator were integrated and tested using a dedicated testbench. The input signal, a 1 MHz sine wave sampled at 120

MHz and represented in Q1.15 fixed-point format, was generated using Python and supplied to the simulation through a text file.

During simulation, Vivado was used to observe the behavior of the system at different stages, including the input signal, decimator output, scaling block output, and interpolator output. The waveform results confirmed that the decimator successfully reduced the sampling frequency from 120 MHz to 40 MHz while preventing aliasing through low-pass filtering. Similarly, the interpolator increased the sampling frequency from 40 MHz to 480 MHz and reconstructed the signal by removing imaging components introduced during upsampling.

The simulation waveforms demonstrated correct data transfer between all processing blocks and verified that the Q1.15 fixed-point format was maintained throughout the design. The generated output samples were exported to text files and further analyzed using Python-based FFT plots. The simulation results confirmed that the implemented design achieved the required sample-rate conversion while preserving the frequency content of the original 1 MHz input signal.

Simulation Steps in Vivado:

1. Create a new Vivado project and add the Verilog source files.
2. Generate the FIR Compiler IPs for decimation and interpolation.
3. Add the testbench and input sample text file.
4. Run Behavioral Simulation.
5. Observe input and output waveforms in the Vivado Waveform Viewer.
6. Export simulation outputs to text files.
7. Analyze the results using Python FFT plots.

Results Obtained:

- Input Sampling Frequency = 120 MHz
- Decimated Sampling Frequency = 40 MHz
- Interpolated Sampling Frequency = 480 MHz
- Input/Output Format = Q1.15 (16-bit fixed-point)
- Correct sample-rate conversion achieved
- No significant aliasing or imaging observed after filtering

This project successfully implemented a multirate digital signal processing system that performs **decimation by 3** and **interpolation by 12** using **51-tap FIR filters**. The design was developed using **Verilog HDL** and simulated in **Xilinx Vivado**, while **Python** was used for generating the input signal, processing simulation data, and performing FFT analysis. The FIR filters effectively removed aliasing and imaging components during sample-rate conversion, ensuring accurate signal reconstruction.

The simulation results and FFT plots verified the correct operation of the system at each processing stage. The sampling frequency was successfully converted from **120 MHz to 40 MHz** through decimation and then increased to **480 MHz** through interpolation while maintaining the **Q1.15 fixed-point format**. This project provided practical experience in FPGA-based DSP design, FIR filter implementation, fixed-point arithmetic, and signal analysis, demonstrating the effectiveness of multirate filtering techniques in modern communication and signal processing applications.

References

- [1] J. G. Proakis and D. G. Manolakis, **Digital Signal Processing: Principles, Algorithms, and Applications**, 4th Edition, Pearson Education, 2007.
- [2] Sanjit K. Mitra, **Digital Signal Processing: A Computer-Based Approach**, 4th Edition, McGraw-Hill Education, 2011.
- [3] Samir Palnitkar, **Verilog HDL: A Guide to Digital Design and Synthesis**, 2nd Edition, Pearson Education, 2003.
- [4] Xilinx Inc., **Vivado Design Suite User Guide**, Xilinx Documentation, Available at: <https://www.xilinx.com>
- [5] Xilinx Inc., **FIR Compiler IP Product Guide (PG149)**, Xilinx Documentation.
- [6] Steven W. Smith, **The Scientist and Engineer's Guide to Digital Signal Processing**, California Technical Publishing, 1997.
- [7] Python Software Foundation, **Python Programming Language Documentation**, Available at: <https://www.python.org>
- [8] NumPy Developers, **NumPy User Guide**, Available at: <https://numpy.org>
- [9] Matplotlib Development Team, **Matplotlib Documentation**, Available at: <https://matplotlib.org>
- [10] U. Meyer-Baese, **Digital Signal Processing with Field Programmable Gate Arrays**, 4th Edition, Springer, 2014.
- [11] FPGA Assignment Specification: *Design and Verification of FIR-Based Decimator and Interpolator Using Xilinx FIR Compiler IP*